

CS2013 - Programación III
Unidad 6: adaptadores
Estructuras básicas: stacks y queues

Ruben Rivas Medina

Octubre 2026

Implementación de los adaptadores. Implementar **utec::stack<T>** y **utec::queue<T>** en los archivos **stack_utec.h** y **queue_utec.h**, respectivamente, utilizando C++20.
Uso de los adaptadores en los ejercicios. Las funciones solicitadas deben manipular los stacks y queues únicamente mediante sus operaciones públicas, sin acceder directamente a su representación interna. Se permite crear stacks y queues auxiliares. Cuando una función recibe un contenedor por valor, trabaja con una copia: puede modificarla para resolver el ejercicio, mientras el contenedor original conserva su contenido y su orden.
Lectura de los ejemplos. Las listas entre corchetes indican el orden de extracción de los elementos: en un stack, de **tope a fondo**; en una queue, de **frente a final**. Por ejemplo, en **[1, 2, 3]**, el primer elemento que se extrae es **1**. Esta notación describe el contenido del contenedor, no el orden en que se insertaron sus elementos.
Los casos de uso asumen que se han incluido los encabezados correspondientes y la declaración **using namespace std;**.

Ejercicio 1: merge_stacks (2 puntos)

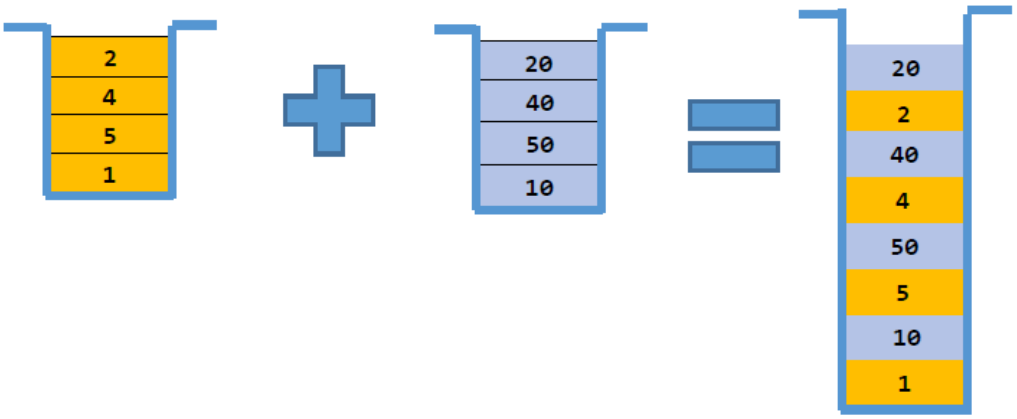
Descripción

Elaborar un template de función que, dados dos stacks, genere un nuevo stack con sus valores intercalados. Comenzar por el tope de **b** y luego tomar el tope de **a**, como muestra el diagrama. Cuando uno se agote, conservar el orden de los valores restantes del otro.

```
template <typename T>
utec::stack<T> merge_stacks(utec::stack<T> a, utec::stack<T> b);
```

Ejemplo

```
De tope a fondo:
a: [1, 2, 3, 4]   b: [10, 20]
Resultado: [10, 1, 20, 2, 3, 4]
Origen:      b  a   b  a  a  a
Al agotarse b, 3 y 4 de a quedan al fondo, en ese orden.
```



Requisitos

- Como muestra la figura, intercalar desde arriba: el resultado, leído de tope a fondo, debe comenzar con el tope de **b**, seguido del tope de **a**, y continuar alternando en ese orden.
- Si los stacks tienen tamaños distintos, intercalar hasta agotar el más corto. Los elementos restantes del stack más largo deben quedar debajo de la parte intercalada, conservando su orden de tope a fondo. Por tanto, la parte inferior del resultado contiene elementos de un solo stack.
- Si uno de los stacks está vacío, el resultado debe conservar el contenido y el orden del otro; si ambos están vacíos, el resultado debe estar vacío.

Ejemplo de uso

```
utec::stack<int> a, b;
for (int x : {4, 3, 2, 1}) a.push(x); // Tope a fondo: [1, 2, 3, 4]
for (int x : {20, 10}) b.push(x);     // Tope a fondo: [10, 20]
auto result = merge_stacks(a, b);
while (!result.empty()) {
    cout << result.top() << " ";
    result.pop();
}
// Salida: 10 1 20 2 3 4
// Los dos ultimos valores (3 y 4) provienen solo de a.
```

Ejercicio 2: join_stacks (2 puntos)

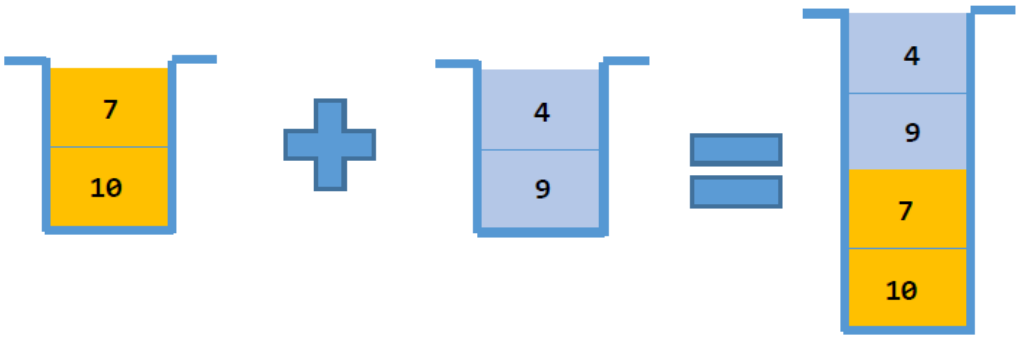
Descripción

Elaborar un template de función que una dos stacks. El resultado debe contener primero todos los elementos de **b**, de tope a fondo, y después todos los de **a**, también de tope a fondo, como muestra el diagrama.

```
template <typename T>
utec::stack<T> join_stacks(utec::stack<T> a, utec::stack<T> b);
```

Ejemplo

```
a: [1, 2, 3]   b: [10, 20]
Resultado: [10, 20, 1, 2, 3]
```



Requisitos

- Conservar los valores repetidos y considerar entradas vacías.

Ejemplo de uso

```
utec::stack<int> a, b;
for (int x : {3, 2, 1}) a.push(x);
for (int x : {20, 10}) b.push(x);
auto result = join_stacks(a, b);
while (!result.empty()) {
    cout << result.top() << " ";
    result.pop();
}
// Salida: 10 20 1 2 3
```

Ejercicio 3: process_text_by_stack (2 puntos)

Descripción

Elaborar una función que procese una secuencia de letras y asteriscos utilizando un stack. Una letra significa un **push** y un asterisco significa un **pop**. Retornar, en orden de extracción, los caracteres retirados del stack; no incluir los que permanezcan almacenados al terminar.

```
string process_text_by_stack(string source);
```

Ejemplos

```
EAS*Y*QUE***ST***IO*N*** -> SYEUQTSAONIE
COMPUTER*****S**CIEN**CE -> RETUPSMNE
```

Requisitos

- Si aparece un asterisco con el stack vacío, ignorarlo.
- La cadena vacía produce una cadena vacía.

Ejemplo de uso

```
string text;
getline(cin, text);
cout << process_text_by_stack(text);
// Entrada: ABC**D*
// Salida: CBD
```

Ejercicio 4: process_text_by_queue (2 puntos)

Descripción

Elaborar una función que procese una secuencia de letras y asteriscos utilizando una queue. Una letra significa un **push** y un asterisco significa un **pop**. Retornar, en orden de extracción, los caracteres retirados del frente; no incluir los que queden pendientes al terminar.

```
string process_text_by_queue(const string& source);
```

Ejemplos

```
EAS*Y*QUE***ST***IO*N*** -> EASYQUESTION
UN*IV*ERS*IDAD**         -> UNIVE
COMPUTER*****S**CIEN**CE -> COMPUTERS
```

Requisitos

- Si aparece un asterisco con la queue vacía, ignorarlo.
- La cadena vacía produce una cadena vacía.

Ejemplo de uso

```
string text;
getline(cin, text);
cout << process_text_by_queue(text);
// Entrada: ABC**D*
// Salida: ABC
```

Ejercicio 5: is_equation_balanced (4 puntos)

Descripción

Dada una ecuación, elaborar una función basada en stack que determine si el balance de separadores es correcto. Cada cierre debe corresponder al último separador abierto que aún no haya sido cerrado.

```
bool is_equation_balanced(string source);
```

Ejemplos

```
((10 + 20) + 30)          -> true
((10 +(30 * 1 + 2))       -> false
(10 + (15 + 4/2 - 5)))    -> false
{ (6 + 20} + 4)           -> false
{ [ (4 - 5) * 3] + 14}    -> true
```

Requisitos

- Validar los pares **()**, **[]** y **{}**; ignorar los demás caracteres.
- Una cadena vacía o sin separadores se considera balanceada.

Ejemplo de uso

```
string text;
getline(cin, text);
cout << boolalpha << is_equation_balanced(text);
```

Ejercicio 6: Historial de navegación con Memento (4 puntos)

Descripción

Implementar un historial de navegación aplicando el patrón **Memento** visto en clases anteriores. El estado del navegador incluye la URL actual y la posición de desplazamiento vertical, representada por un entero no negativo.
Distribuir las responsabilidades entre tres clases:

- **Browser** (**Originator**) mantiene el estado actual, crea instantáneas y restaura su estado a partir de ellas.
- **BrowserMemento** (**Memento**) encapsula una copia independiente de la URL y de la posición de desplazamiento. Solo **Browser** puede crear e inspeccionar estas instantáneas.
- **BrowserHistory** (**Caretaker**) administra dos stacks de instantáneas para retroceder y avanzar. Puede almacenarlas y pasarlas a **Browser**, pero no consultar ni modificar su contenido.

Implementar las siguientes interfaces. Agregar los atributos privados necesarios; los stacks pertenecen a **BrowserHistory**.

```
class BrowserMemento {
    friend class Browser;
    string url_;
    int scroll_;
    BrowserMemento(const string& url, int scroll);
public:
    BrowserMemento(const BrowserMemento&) = default;
    BrowserMemento& operator=(const BrowserMemento&) = default;
};

class Browser {
public:
    explicit Browser(const string& initial_url);
    void visit(const string& url);
    void set_scroll(int position);
    string current() const;
    int scroll() const;
    BrowserMemento save() const;
    void restore(const BrowserMemento& state);
};

class BrowserHistory {
public:
    explicit BrowserHistory(Browser& browser);
    void visit(const string& url);
    bool back();
    bool forward();
};
```

Requisitos

- Usar **utec::stack<BrowserMemento>** para ambos historiales. **BrowserHistory** debe trabajar con una referencia al navegador recibido en su constructor, cuya vida útil debe abarcar la del historial. Inicialmente, ambos stacks están vacíos.
- **Browser::visit()** cambia la URL y establece el desplazamiento en **0**. El constructor también comienza con desplazamiento **0**. **set_scroll()** actualiza la posición actual sin crear entradas en el historial.
- **BrowserHistory::visit()** guarda una instantánea del estado actual en el stack de retroceso, vacía el de avance y solicita al navegador visitar la nueva URL.
- **back()** guarda el estado actual en el stack de avance, restaura la instantánea del tope del stack de retroceso y la retira de ese stack. **forward()** realiza la operación inversa. Ambas operaciones restauran la URL y el desplazamiento.
- **back()** y **forward()** retornan **true** si restauran una instantánea. Si el stack de origen está vacío, retornan **false** sin modificar el navegador ni los historiales.
- Las instantáneas conservan su estado aunque el navegador cambie después. No exponer getters ni campos públicos en **BrowserMemento**, ni declarar **BrowserHistory** como su clase amiga.
- Las URL son cadenas no vacías y las posiciones son no negativas; no es necesario validar estas precondiciones. Se permiten visitas repetidas.

Ejemplo de uso

```
Browser browser("inicio");
BrowserHistory history(browser);
browser.set_scroll(120);
history.visit("cursos");       // Guarda ("inicio", 120).
browser.set_scroll(450);
history.visit("adaptadores"); // Guarda ("cursos", 450).

cout << boolalpha << history.back() << "\n"; // true
cout << browser.current() << " " << browser.scroll() << "\n";
// cursos 450: se restauran ambos datos de la instantanea.

cout << history.forward() << "\n"; // true
cout << browser.current() << " " << browser.scroll() << "\n";
// adaptadores 0

history.back();
history.visit("evaluaciones"); // Descarta el historial de avance.
cout << history.forward() << "\n"; // false
cout << browser.current() << " " << browser.scroll() << "\n";
// evaluaciones 0
```

Ejercicio 7: Validación de etiquetas (4 puntos)

Descripción

Elaborar una función basada en stack que determine si las etiquetas de un documento están correctamente anidadas. Cada etiqueta de cierre debe coincidir con la última etiqueta de apertura pendiente. Ignorar el texto fuera de las etiquetas.

```
bool validate_tags(const string& source);
```

Ejemplos

```
<libro><titulo>Hola</titulo></libro> -> true
<a><b></a></b>                       -> false
<a></a><b></b>                       -> true
<a><b></b>                           -> false
</a>                                 -> false
```

Requisitos

- Usar **utec::stack<string>**. Distinguir mayúsculas de minúsculas.
- Solo se admiten **<nombre>** y **</nombre>**, con nombres formados por una o más letras ASCII. No se admiten atributos, espacios internos, comentarios ni etiquetas autocerradas.
- Todo **<** inicia una etiqueta y todo **>** debe cerrarla. Retornar **false** ante sintaxis inválida o etiquetas pendientes al finalizar.
- La cadena vacía y el texto sin etiquetas son válidos. No exigir una única etiqueta raíz.

Ejemplo de uso

```
string text;
getline(cin, text);
cout << boolalpha << validate_tags(text);
// Entrada: <curso><tema>Stacks</tema></curso>
// Salida: true
```

Ejercicio 8: evaluate_postfix (4 puntos)

Descripción

Elaborar una función basada en stack que evalúe una expresión en notación postfija. Los tokens se separan por uno o más espacios y pueden ser enteros con signo o los operadores binarios **+**, **-**, ***** y **/**.

```
int evaluate_postfix(const string& source);
```

Ejemplos

```
10 20 + 3 * -> 90
20 5 / 2 -  -> 2
-3 4 * 2 +  -> -10
8 3 -       -> 5
2 3 4 + *   -> 14 (operadores consecutivos: valido)
2 3 + *     -> invalid_argument (a * le falta un operando)
10 +        -> invalid_argument (falta un operando)
10 20       -> invalid_argument (sobran operandos)
10 dos +    -> invalid_argument (token invalido: dos)
```

En **10 +**, el operador **+** necesita dos operandos, pero el stack solo contiene **10**. La función debe lanzar **invalid_argument** antes de intentar extraer un segundo operando. En este caso no se retorna un resultado numérico.

Requisitos

- Usar **utec::stack<int>**. Procesar los tokens de izquierda a derecha: apilar cada entero; ante un operador, verificar que haya al menos dos valores disponibles, extraerlos y apilar el resultado. El primer operando extraído es el derecho; el segundo es el izquierdo.
- Se permiten operadores consecutivos: cada uno debe encontrar al menos dos valores disponibles en el stack. En **2 3 4 + ***, **+** combina **3** y **4**, y ***** combina **2** y **7**. En **2 3 + ***, después de **+** solo queda **5**, por lo que ***** provoca **invalid_argument**.
- La división es entera con truncamiento hacia cero, como en C++.
- Lanzar **invalid_argument** si la expresión está vacía, contiene tokens inválidos, faltan operandos o no queda exactamente un valor al terminar.
- Lanzar **domain_error** ante división entre cero. Suponer que los enteros y todos los resultados intermedios son representables como **int**.

Ejemplo de uso

```
string text;
getline(cin, text);
cout << evaluate_postfix(text);
// Entrada: 10 20 + 3 *
// Salida: 90
```